Rising Waters - Flood Prediction System using Machine Learning

Project Description:

Overview

Rising Waters is a web-based Flood Prediction System developed using Machine Learning to assist in the early prediction of flood occurrences based on weather-related parameters. Floods are among the most devastating natural disasters, causing significant damage to lives, property, agriculture, and infrastructure. Accurate early prediction allows authorities and communities to take preventive measures before severe flooding occurs.

This project uses historical weather data collected from a Kaggle dataset to train multiple machine learning models. The dataset contains important environmental features such as Cloud Cover, Annual Rainfall, Jan–Feb Rainfall, Mar–May Rainfall, and Jun–Sep Rainfall. These features are analyzed to determine whether flood conditions are likely to occur.

Several machine learning algorithms including Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and XGBoost were trained and evaluated. After comparing their performance using evaluation metrics such as Accuracy Score, Confusion Matrix, and Classification Report, the XGBoost model achieved the best prediction accuracy and was selected as the final model.

The trained model is integrated into a Flask web application, allowing users to enter weather parameters through a simple web interface. Once the input is submitted, the application preprocesses the data using a saved StandardScaler before passing it to the trained XGBoost model. The prediction is then displayed as either **Flood** or **No Flood**.

The project demonstrates how Machine Learning can be applied in disaster management to improve decision-making, reduce response time, and provide reliable flood predictions using historical weather information.

Objectives

The primary objectives of this project are:

- Develop an intelligent flood prediction system using Machine Learning.
- Analyze historical weather data to identify flood patterns.
- Compare multiple machine learning algorithms and select the most accurate model.
- Build a user-friendly web application using Flask.
- Provide quick and reliable flood predictions based on user input.
- Improve disaster preparedness by supporting early flood warning systems.

Problem Statement

Traditional flood prediction methods often depend on manual observation, delayed weather analysis, or complex forecasting systems that may not provide timely warnings. These limitations can reduce the effectiveness of disaster preparedness and increase the risk of economic loss and human casualties.

The project addresses this problem by creating an automated Machine Learning-based Flood Prediction System capable of generating accurate predictions using weather parameters. The

system minimizes manual effort while providing faster and more reliable prediction results.

Key Features

- Machine Learning-based flood prediction.
- Historical weather data analysis.
- Data preprocessing using StandardScaler.
- Comparison of multiple classification algorithms.
- XGBoost-based prediction model.
- Flask-powered web application.
- Interactive user interface developed using HTML, CSS, and JavaScript.
- Instant Flood / No Flood prediction.
- Simple and easy-to-use interface.
- Reusable trained model using Joblib.

Technologies Used

Component	Technology
Programming Language	Python
Machine Learning	Scikit-learn, XGBoost
Frontend	HTML, CSS, JavaScript
Backend	Flask
Dataset	Kaggle Flood Prediction Dataset
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Model Storage	Joblib
Development Environment	Visual Studio Code, Jupyter Notebook
Version Control	Git, GitHub

Real-World Applications

The developed Flood Prediction System can be applied in several real-world scenarios, including:

- Disaster Management Authorities
- Weather Monitoring Centers
- Government Disaster Response Agencies
- Agricultural Planning
- Smart City Infrastructure
- Environmental Research
- Community Flood Warning Systems

The system can be further enhanced by integrating real-time weather APIs and cloud deployment to provide live flood prediction services.

Project Workflow

The overall workflow of the system follows these steps:

1. Collect historical flood dataset.
2. Perform data preprocessing and cleaning.
3. Split the dataset into training and testing data.
4. Train multiple machine learning models.
5. Evaluate model performance.
6. Select the best-performing model (XGBoost).
7. Save the trained model and scaler using Joblib.
8. Develop a Flask web application.
9. Accept weather parameters from users.
10. Predict flood occurrence.
11. Display prediction results through the web interface.

Technical Architecture:

Description

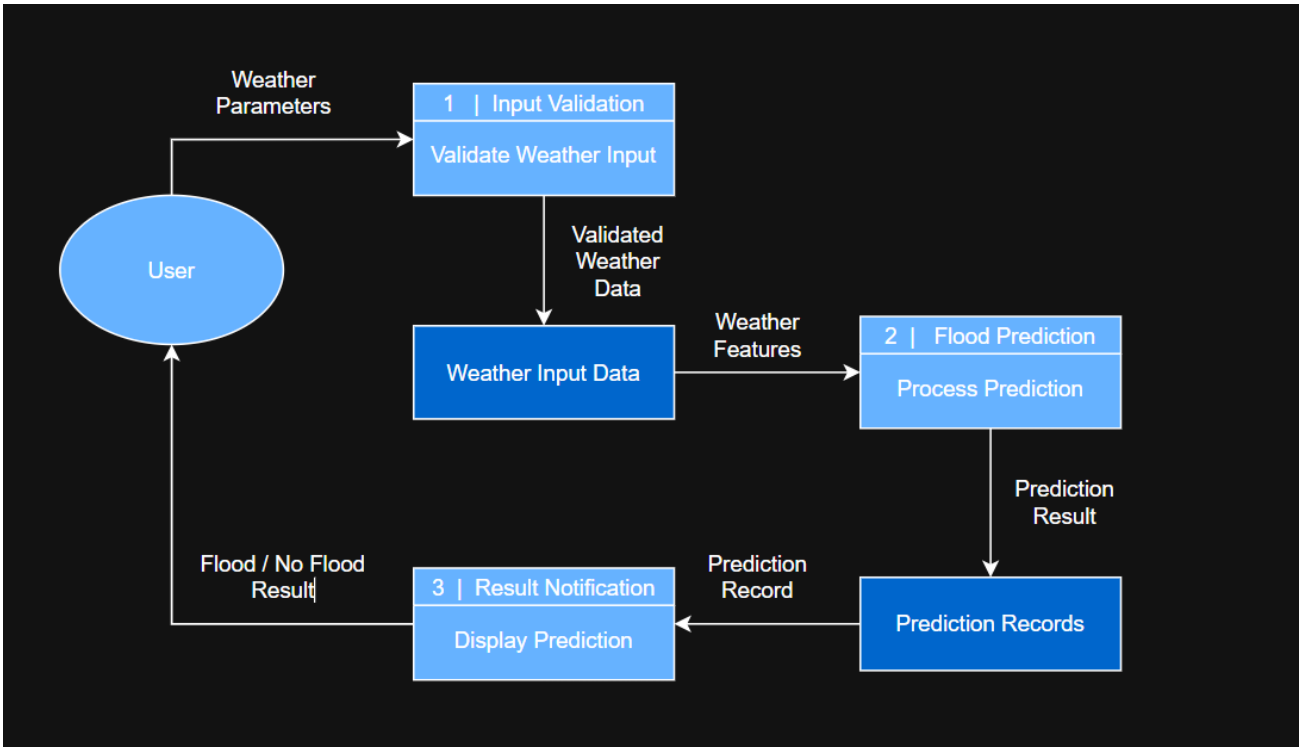
The **Flood Prediction System** follows a modular three-tier architecture consisting of the **Presentation Layer**, **Application Layer**, and **Data Layer**. This architecture separates user interaction, business logic, and data management, making the system easy to understand, maintain, and extend.

The frontend provides a simple and responsive interface where users enter weather parameters such as Cloud Cover and Rainfall values. The backend, developed using the Flask framework, processes user requests, validates the entered values, and forwards the processed data to the Machine Learning prediction module.

The Machine Learning module loads the previously trained XGBoost model and StandardScaler using Joblib. The scaler preprocesses the user input before passing it to the prediction model. Based on the processed features, the model predicts whether flood conditions are likely to occur and returns the result to the Flask application. Finally, the prediction is displayed to the user through the web interface.

This layered architecture keeps the presentation, prediction logic, and storage components independent, improving code maintainability and making future enhancements easier.

System Architecture



The architecture consists of the following layers:

Presentation Layer

⇩

HTML + CSS + JavaScript

⇩

Flask Backend

⇩

Input Validation

⇩

StandardScaler

⇩

XGBoost Model

⇩

Prediction Result

⇩

Flood / No Flood

Architecture Components

1. Presentation Layer

The Presentation Layer provides the user interface of the application. It is responsible for collecting weather-related parameters from the user and displaying the prediction result in an easy-to-understand format.

Technologies Used:

- HTML
- CSS
- JavaScript

Responsibilities:

- Display web pages.
- Accept user inputs.
- Show prediction results.

2. Application Layer

The Application Layer contains the core logic of the system. It is responsible for processing user requests, validating input values, preprocessing the data, and interacting with the trained Machine Learning model.

Technologies Used:

- Python
- Flask

Responsibilities:

- Receive user requests.
- Validate input values.
- Invoke the prediction model.
- Return prediction results.

3. Machine Learning Layer

The Machine Learning Layer is responsible for generating flood predictions based on historical weather patterns.

Technologies Used:

- XGBoost
- Scikit-learn
- Joblib

Responsibilities:

- Load the trained model.
- Load the saved StandardScaler.
- Preprocess input data.
- Generate predictions.

4. Data Layer

The Data Layer stores the resources required for prediction.

Resources Stored:

- Kaggle Flood Prediction Dataset
- Trained XGBoost Model
- StandardScaler Object

Technologies Used:

- Excel/CSV Dataset
- Joblib

Data Flow

The overall data flow of the system is as follows:

1. The user enters weather parameters through the web application.
2. Flask receives the submitted request.
3. Input values are validated.
4. The StandardScaler preprocesses the input features.
5. The XGBoost model predicts the flood status.
6. The prediction result is returned to Flask.
7. Flask displays either **Flood** or **No Flood** on the result page.

Advantages of the Architecture

The selected architecture provides several advantages:

- Simple and easy to understand.
- Modular and maintainable design.
- Fast prediction response.
- Easy integration of new Machine Learning models.
- Separation of frontend, backend, and prediction logic.
- Supports future enhancements such as cloud deployment and real-time weather API integration.

Technologies Used

Layer	**Technology**
Frontend	HTML, CSS, JavaScript
Backend	Flask
Machine Learning	XGBoost, Scikit-learn
Data Processing	Pandas, NumPy
Model Storage	Joblib
Dataset	Kaggle Flood Prediction Dataset

Architecture Summary

The proposed architecture provides an efficient framework for implementing a Machine Learning-based Flood Prediction System. By separating user interaction, application logic, and prediction functionality into independent layers, the system becomes scalable, reusable, and easy to maintain. The Flask framework enables seamless communication between the web interface and the Machine Learning model, while XGBoost provides accurate flood predictions based on weather parameters. The architecture also allows future integration of cloud services, real-time weather APIs, and advanced disaster management features.

Prerequisites:

Before developing and executing the **Flood Prediction System**, several software tools, libraries, and hardware components are required. These prerequisites ensure that the application runs correctly, the machine learning model functions efficiently, and the Flask web application can be executed without issues.

The project was developed using Python and Flask while utilizing machine learning libraries such as Scikit-learn and XGBoost for model development. Historical flood data obtained from Kaggle was used for training and evaluation.

Software Requirements

The following software must be installed before running the application.

Software	Purpose
Python 3.10 or above	Programming language used for development
Visual Studio Code	Source code editor
Jupyter Notebook	Dataset analysis and model training
Git	Version control
GitHub	Project repository management
Google Chrome / Microsoft Edge	Running the web application

Hardware Requirements

The minimum hardware specifications required are:

Component	Specification
Processor	Intel Core i5 or higher
RAM	8 GB
Storage	256 GB SSD or higher
Internet	Required for downloading libraries and project dependencies

Python Libraries

The following Python libraries are required:

Library	Purpose
Flask	Web application framework
Pandas	Data manipulation
NumPy	Numerical computations
Scikit-learn	Machine learning utilities
XGBoost	Flood prediction model
Matplotlib	Data visualization
Seaborn	Statistical visualization
Joblib	Saving and loading trained models

Install them using:

pip install -r requirements.txt

or

pip install flask pandas numpy scikit-learn xgboost matplotlib seaborn joblib

Dataset

The project uses the **Kaggle Flood Prediction Dataset**.

Dataset characteristics:

- Historical weather observations
- Weather-related numerical features
- Flood classification labels
- Excel/CSV format
- Suitable for supervised machine learning

The primary input features include:

- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall

Target Variable:

- Flood (Yes/No)

Development Environment

The project was developed using:

- Python
- Visual Studio Code
- Jupyter Notebook

- Flask Framework
- GitHub

The trained machine learning model and scaler were stored using Joblib for later use in the Flask application.

Project Folder Structure

The project follows the structure below:

```
Flood_Prediction_System/
│
├── app.py
├── Flood_Prediction.ipynb
├── floods.save
├── transform.save
├── requirements.txt
├── README.md
├── dataset.xlsx
│
├── templates/
│   ├── home.html
│   ├── index.html
│   ├── chance.html
│   └── no_chance.html
│
├── static/
│   ├── main.css
│   ├── main.js
│   └── flood.jpg
```

Installation Steps

Follow these steps to set up the project.

Step 1 – Install Python

Download and install Python 3.10 or later from the official Python website.

Step 2 – Download the Project

Clone the repository using Git.

git clone <repository-link>

or download the ZIP file and extract it.

Step 3 – Install Dependencies

Open the project folder.

Run:

pip install -r requirements.txt

Step 4 – Verify Required Files

Ensure the following files are available:

- floods.save
- transform.save
- dataset.xlsx
- app.py

Step 5 – Run the Application

Execute:

python app.py

Step 6 – Open the Browser

Visit

http://127.0.0.1:5000/

Step 7 – Test the Application

Enter:

- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall

Click **Predict Flood**.

The application displays either:

- Flood
- No Flood

Output Verification

The setup is considered successful if:

- Flask server starts without errors.
- The web application opens successfully.
- Users can submit weather parameters.

- Prediction results are displayed correctly.
- No runtime exceptions occur during prediction.

Conclusion

The prerequisites outlined above provide a complete development environment for the Flood Prediction System. Installing the required software, libraries, and dependencies ensures smooth execution of the machine learning model and Flask application. With the proper setup in place, the system can accurately predict flood conditions using historical weather data and provide users with quick and reliable results.

Project Workflow:

Milestone 1: Dataset Collection and Data Preprocessing

Introduction

The first milestone focuses on collecting a suitable dataset and preparing it for machine learning model development. Data preprocessing is one of the most important stages of any machine learning project because the quality of the dataset directly influences the prediction accuracy of the final model.

For this project, a historical **Flood Prediction Dataset** obtained from Kaggle was used. The dataset contains weather-related parameters that influence flood occurrence. Before training the machine learning models, the dataset was carefully inspected, cleaned, and transformed into a format suitable for model training.

The preprocessing stage included loading the dataset, exploring its structure, checking for missing values, analyzing feature distributions, selecting important features, splitting the data into training and testing sets, and applying feature scaling using StandardScaler.

Activity 1.1: Collect the Flood Prediction Dataset

The first activity involved collecting a reliable historical flood prediction dataset from Kaggle. The selected dataset contains numerical weather attributes used to classify whether flood conditions are likely to occur.

Dataset Source

Source: Kaggle Flood Prediction Dataset

Dataset Features

The dataset contains the following important features:

- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall
- Flood (Target Variable)

The **Flood** column represents the prediction target, where the model learns to classify whether flooding is expected based on the supplied weather parameters.

Activity 1.2: Load the Dataset

The dataset was loaded into the Jupyter Notebook using the Pandas library.

The following operations were performed:

- Import the dataset.
- Display the first few records.
- Verify the number of rows and columns.
- Identify data types.

This helped in understanding the overall structure of the dataset before preprocessing.

```python
# Displaying first five rows

dataset.head()
```

Python

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune	sub	flood
0	29	70	30	3248.6	73.4	386.2	2122.8	666.1	274.866667	649.9	0
1	28	75	40	3326.6	9.3	275.7	2403.4	638.2	130.300000	256.4	1
2	28	75	42	3271.2	21.7	336.3	2343.0	570.1	186.200000	308.9	0
3	29	71	44	3129.7	26.7	339.4	2398.2	365.3	366.066667	862.5	0
4	31	74	40	2741.6	23.4	378.5	1881.5	458.1	283.400000	586.9	0

Activity 1.3: Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed to understand the characteristics of the dataset before model training.

The following analyses were carried out:

- Display dataset information.
- Generate statistical summaries.
- Check feature distributions.
- Analyze relationships among variables.
- Visualize important data patterns.

Data visualization techniques such as histograms, correlation heatmaps, and distribution plots were used to gain insights into the dataset.

```python
# Information about the dataset
dataset.info()
```

Python

```
<class 'pandas.DataFrame'>
RangeIndex: 115 entries, 0 to 114
Data columns (total 11 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   Temp         115 non-null    int64
 1   Humidity     115 non-null    int64
 2   Cloud Cover  115 non-null    int64
 3   ANNUAL       115 non-null    float64
 4   Jan-Feb      115 non-null    float64
 5   Mar-May      115 non-null    float64
 6   Jun-Sep      115 non-null    float64
 7   Oct-Dec      115 non-null    float64
 8   avgjune      115 non-null    float64
 9   sub          115 non-null    float64
 10  flood        115 non-null    int64
dtypes: float64(7), int64(4)
memory usage: 10.0 KB
```

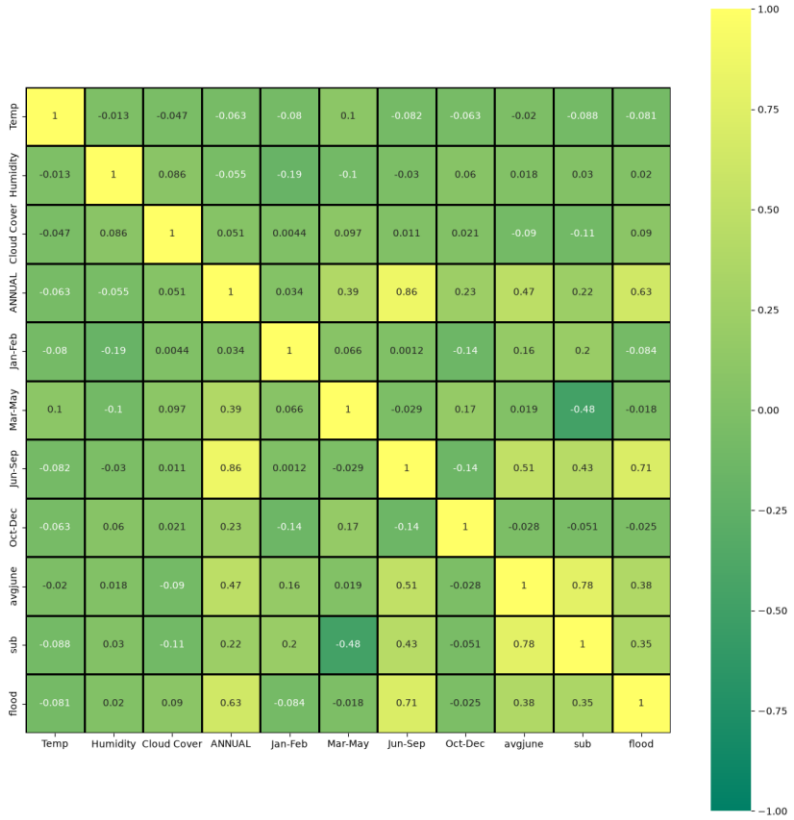
```
# Statistical summary
dataset.describe()
```

Python

	Temp	Humidity	Cloud Cover	ANNUAL	Jan-Feb	Mar-May	Jun-Sep	Oct-Dec	avgjune
count	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000	115.000000
mean	29.600000	73.852174	36.286957	2925.487826	27.739130	377.253913	2022.840870	497.636522	218.100870
std	1.122341	2.947623	4.330158	422.112193	22.361032	151.091850	386.254397	129.860643	62.547597
min	28.000000	70.000000	30.000000	2068.800000	0.300000	89.900000	1104.300000	166.600000	65.600000
25%	29.000000	71.000000	32.500000	2627.900000	10.250000	276.750000	1768.850000	407.450000	179.666667
50%	30.000000	74.000000	36.000000	2937.500000	20.500000	342.000000	1948.700000	501.500000	211.033333
75%	31.000000	76.000000	40.000000	3164.100000	41.600000	442.300000	2242.900000	584.550000	263.833333
max	31.000000	79.000000	44.000000	4257.800000	98.100000	915.200000	3451.300000	823.300000	366.066667

Correlation Heatmap:

A correlation heatmap was generated to visualize the relationship between different weather parameters in the dataset. It helped identify positive and negative correlations among the features and provided valuable insights for feature selection before model training.



Activity 1.4: Handle Missing Values

The dataset was examined for missing or null values.

Using Pandas functions, each column was checked to determine whether incomplete records existed.

Observation:

- No missing values were found in the dataset.

Since the dataset was complete, no imputation or data replacement techniques were required.

```
# Checking Missing Values

dataset.isnull().sum()
```

✓ 0.0s Python

```
Temp          0
Humidity      0
Cloud Cover   0
ANNUAL        0
Jan-Feb       0
Mar-May       0
Jun-Sep       0
Oct-Dec       0
avgjune       0
sub           0
flood         0
dtype: int64
```

Activity 1.5: Feature Selection

Relevant weather parameters were selected as input features for the prediction model.

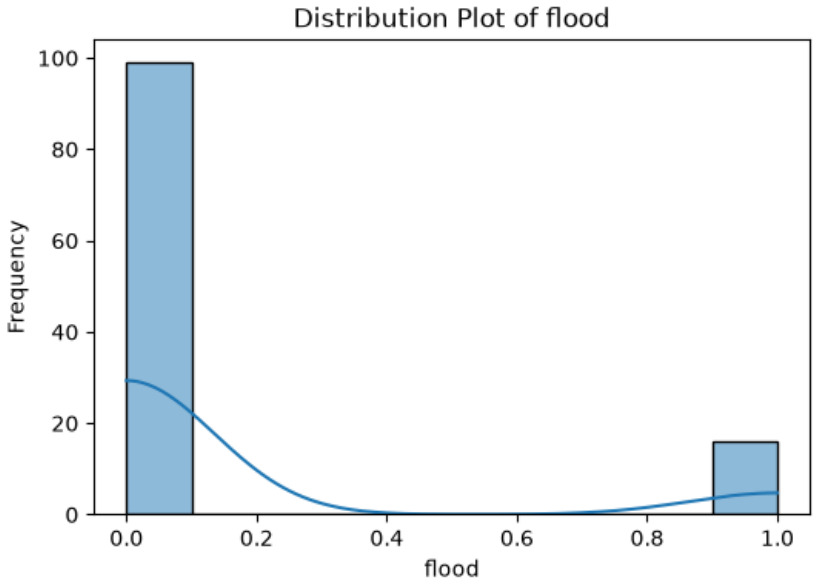
Selected Input Features

- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall

Target Variable

- Flood

Selecting only relevant features improves model performance and reduces unnecessary complexity.



Activity 1.6: Train-Test Split

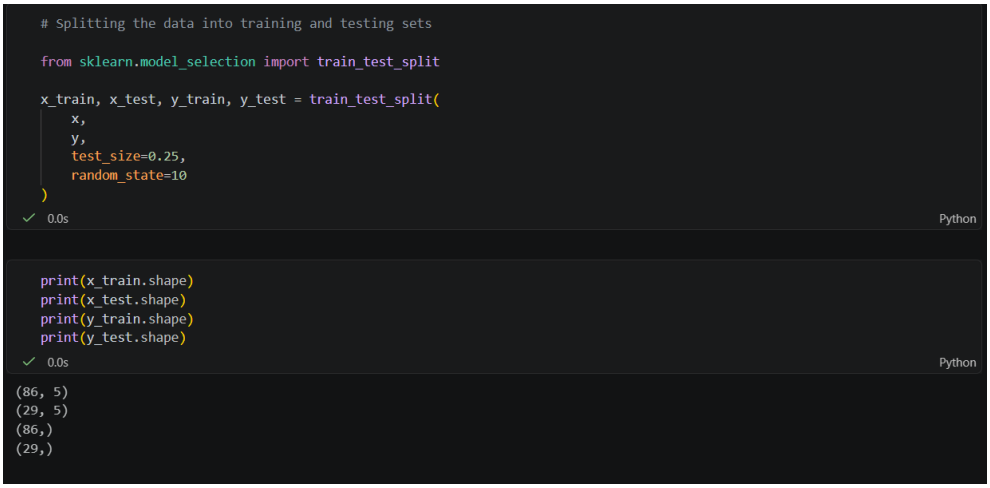
After preprocessing, the dataset was divided into training and testing datasets.

The data was split into:

- Training Data: 80%
- Testing Data: 20%

The training dataset was used to train the machine learning models, while the testing dataset was

reserved for evaluating model performance on unseen data.



Activity 1.7: Feature Scaling

Machine learning algorithms often perform better when numerical features are standardized.

The StandardScaler from Scikit-learn was used to normalize the selected features.

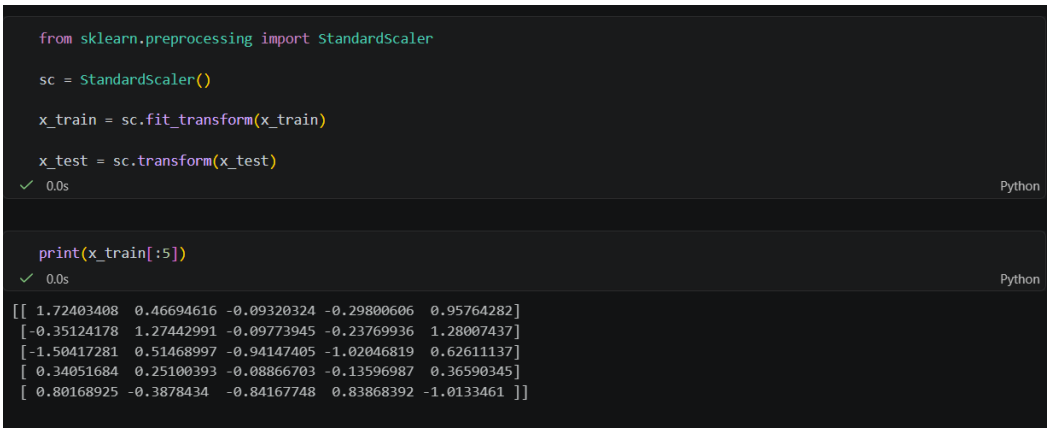
Benefits of feature scaling include:

- Faster model convergence.
- Improved prediction accuracy.
- Equal contribution of numerical features.

The fitted scaler was saved using Joblib as:

- transform.save

This saved scaler is later loaded by the Flask application during prediction.



Milestone 1 Outcome

At the end of this milestone:

- Dataset successfully collected.
- Data loaded into Python.
- Dataset explored using EDA.
- Missing values verified.
- Relevant features selected.
- Dataset split into training and testing sets.

- Features standardized using StandardScaler.

The processed dataset became ready for machine learning model development.

Milestone 1 Summary

The first milestone established the foundation for the Flood Prediction System by preparing a clean and well-structured dataset. Through preprocessing and feature engineering, the data was transformed into a suitable format for machine learning algorithms. These steps significantly contributed to improving the prediction accuracy and reliability of the final system.

Milestone 2 – Machine Learning Model Development

Introduction

The second milestone focuses on developing and evaluating multiple machine learning models for flood prediction. After completing data preprocessing, several supervised classification algorithms were trained using the processed dataset to determine which model could predict flood occurrence with the highest accuracy.

Different algorithms perform differently depending on the characteristics of the dataset. Therefore, instead of relying on a single algorithm, four machine learning models were implemented and compared:

- Decision Tree
- Random Forest
- K-Nearest Neighbors (KNN)
- XGBoost

Each model was evaluated using standard machine learning evaluation metrics such as Accuracy Score, Confusion Matrix, and Classification Report. The model with the highest performance was selected for deployment in the Flask web application.

Activity 2.1: Decision Tree Model

The Decision Tree algorithm was implemented as the first classification model.

A Decision Tree predicts the target class by repeatedly splitting the dataset based on feature values. Each internal node represents a decision, while each leaf node represents the predicted class.

The implementation involved:

- Initializing the DecisionTreeClassifier.
- Training the model using the training dataset.
- Generating predictions on the testing dataset.
- Evaluating performance using multiple metrics.

The model's performance was analyzed using:

- Accuracy Score
- Confusion Matrix
- Classification Report

```
Decision Tree Accuracy:
0.9655172413793104

Confusion Matrix:
[[26  0]
 [ 1  2]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        26
           1       1.00      0.67      0.80         3

    accuracy                           0.97        29
   macro avg       0.98      0.83      0.89        29
weighted avg       0.97      0.97      0.96        29
```

Activity 2.2: Random Forest Model

The Random Forest algorithm was implemented as the second model.

Random Forest combines multiple Decision Trees to improve prediction accuracy and reduce overfitting. Each tree makes an independent prediction, and the final prediction is determined by majority voting.

The implementation steps included:

- Initializing the RandomForestClassifier.
- Training the classifier.
- Predicting flood classes.
- Evaluating model performance.

Evaluation metrics included:

- Accuracy Score
- Confusion Matrix
- Classification Report

The Random Forest model demonstrated better generalization compared to a single Decision Tree.

```
========== RANDOM FOREST MODEL ==========
Accuracy: 0.9655172413793104

Confusion Matrix:
[[26  0]
 [ 1  2]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        26
           1       1.00      0.67      0.80         3

    accuracy                           0.97        29
   macro avg       0.98      0.83      0.89        29
weighted avg       0.97      0.97      0.96        29
```

Activity 2.3: K-Nearest Neighbors (KNN)

The KNN algorithm classifies new observations based on the nearest neighboring data points.

The following steps were performed:

- Initialize KNeighborsClassifier.
- Train the classifier.
- Predict testing samples.
- Evaluate prediction accuracy.

KNN provided acceptable prediction results but required more computation during prediction because it compares each new sample with the training dataset.

```
========== KNN MODEL ==========
Accuracy: 0.896551724137931

Confusion Matrix:
[[23  3]
 [ 0  3]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      0.88      0.94        26
           1       0.50      1.00      0.67         3

    accuracy                           0.90        29
   macro avg       0.75      0.94      0.80        29
weighted avg       0.95      0.90      0.91        29
```

Activity 2.4: XGBoost Model

The XGBoost algorithm was implemented as the final machine learning model.

XGBoost is an advanced boosting algorithm that builds multiple decision trees sequentially. Each new tree attempts to correct the prediction errors of previous trees, resulting in high prediction accuracy.

Implementation included:

- Initializing the XGBClassifier.
- Training the model.
- Predicting test data.
- Evaluating performance.

Evaluation metrics:

- Accuracy Score
- Confusion Matrix
- Classification Report

Among all the tested algorithms, XGBoost achieved the best prediction accuracy and was selected as the final model for deployment.

```
========== XGBOOST MODEL ==========
Accuracy: 0.9655172413793104

Confusion Matrix:
[[26  0]
 [ 1  2]]

Classification Report:
              precision    recall  f1-score   support

           0       0.96      1.00      0.98        26
           1       1.00      0.67      0.80         3

    accuracy                           0.97        29
   macro avg       0.98      0.83      0.89        29
weighted avg       0.97      0.97      0.96        29
```

Activity 2.5: Model Comparison

After training all four algorithms, their performances were compared.

Model	Evaluation Metrics
Decision Tree	Accuracy, Confusion Matrix, Classification Report
Random Forest	Accuracy, Confusion Matrix, Classification Report
KNN	Accuracy, Confusion Matrix, Classification Report
XGBoost	Accuracy, Confusion Matrix, Classification Report

The comparison showed that XGBoost provided the highest overall prediction performance for the flood dataset.

```
========== MODEL COMPARISON ==========
Decision Tree  : 0.9655172413793104
Random Forest : 0.9655172413793104
KNN           : 0.896551724137931
XGBoost       : 0.9655172413793104
```

Activity 2.6: Model Selection

Based on the comparison results, XGBoost was selected because it:

- Achieved the highest prediction accuracy.
- Reduced overfitting.
- Produced reliable classification results.
- Generalized well on unseen data.

The trained model was then prepared for deployment.

Activity 2.7: Saving the Model

The final trained XGBoost model was saved using Joblib.

Saved files:

- floods.save — trained XGBoost model.

- transform.save — StandardScaler object.

Saving the model allows the Flask application to make predictions without retraining the model every time.

```
import joblib

# Save the trained model
joblib.dump(xgb, "floods.save")

# Save the scaler
joblib.dump(sc, "transform.save")

print("Model and Scaler saved successfully!")
```

✓ 0.0s Python

```
Model and Scaler saved successfully!
```

Model Evaluation Metrics

The following evaluation metrics were used throughout the project:

Accuracy Score

Measures the percentage of correctly classified samples.

Confusion Matrix

Shows:

- True Positives
- True Negatives
- False Positives
- False Negatives

It provides a detailed understanding of prediction performance.

Classification Report

The classification report includes:

- Precision
- Recall
- F1-Score
- Support

These metrics provide a comprehensive evaluation of the prediction model.

Milestone 2 Outcome

At the end of this milestone:

- Four machine learning algorithms were implemented.
- Each model was trained successfully.
- Performance evaluation was completed.
- XGBoost was selected as the best-performing model.
- The trained model and scaler were saved for deployment.

Milestone 2 Summary

This milestone established the predictive capability of the Flood Prediction System. By comparing multiple machine learning algorithms, the project ensured that the most suitable model was selected for deployment. XGBoost demonstrated superior performance and became the core prediction engine of the system.

Milestone 3 – Flask Application Development

Introduction

After selecting the best-performing machine learning model, the next step was to integrate it into a web application. The Flask framework was chosen because it is lightweight, easy to develop, and well-suited for deploying machine learning models.

The Flask application acts as the communication layer between the user interface and the prediction model. It receives weather parameters entered by the user, preprocesses the data using the saved StandardScaler, generates predictions using the trained XGBoost model, and displays the result through a web page.

This milestone focused on developing the application's backend logic, defining routes, loading the trained model, and implementing prediction functionality.

Activity 3.1: Setting Up the Flask Application

The first step involved creating the Flask project structure.

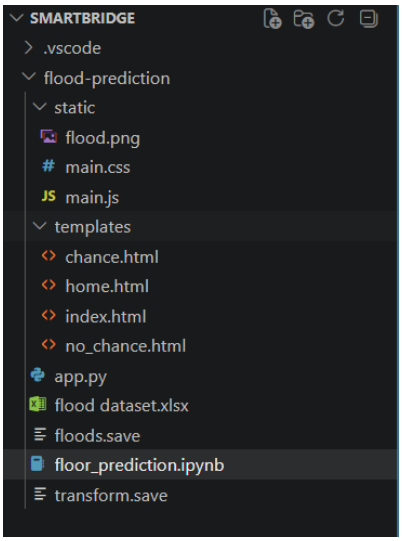
The project includes:

- app.py
- templates/
- static/
- Trained model (floods.save)
- Saved scaler (transform.save)

The Flask framework was initialized using:

app = Flask(__name__)

This creates the main Flask application responsible for handling all user requests.



```
app = Flask(__name__)
```

Activity 3.2: Loading the Trained Model

Instead of training the model every time the application starts, the trained model and scaler were loaded using Joblib.

The following files were loaded:

- floods.save
- transform.save

Benefits:

- Faster prediction.
- Reduced startup time.
- No need for retraining.

```
model = joblib.load("floods.save")
scaler = joblib.load("transform.save")
```

Activity 3.3: Creating Flask Routes

Several routes were defined within the application.

Home Route

The home route loads the landing page.

/

Purpose:

- Display the welcome page.
- Allow users to navigate to the prediction page.

```
@app.route("/")
def home():
    return render_template("home.html")
```

Prediction Page

/Predict

Purpose:

- Display the weather parameter input form.

```
@app.route("/predict", methods=["GET"])
def predict_page():
    return render_template("index.html")
```

Prediction Route

/predict

Purpose:

- Receive user input.
- Process weather parameters.
- Generate flood prediction.

- Display the prediction result.

```
@app.route("/predict", methods=["POST"])
def predict():
```

Activity 3.4: Processing User Input

The application accepts five weather-related parameters:

- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall

These values are collected using Flask's request.form object.

The received values are converted into numerical format before prediction.

Activity 3.5: Data Preprocessing

Before prediction, the user input is transformed into a DataFrame with the same feature names used during training.

The saved StandardScaler preprocesses the input features to maintain consistency between training and prediction.

Benefits:

- Improved prediction accuracy.
- Consistent feature scaling.
- Better model performance.

```
data = pd.DataFrame(
    [[cloud_cover, annual, jan_feb, mar_may, jun_sep]],
    columns=[
        "Cloud Cover",
        "ANNUAL",
        "Jan-Feb",
        "Mar-May",
        "Jun-Sep"
    ]
)

data = scaler.transform(data)
```

Activity 3.6: Flood Prediction

After preprocessing, the scaled input is passed to the trained XGBoost model.

The prediction process consists of:

1. Receive processed features.
2. Generate prediction.
3. Determine prediction class.

Possible outputs:

- Flood

- No Flood

```
prediction = model.predict(data)

if prediction[0] == 1:
    return render_template("chance.html")
else:
    return render_template("no_chance.html")
```

Activity 3.7: Displaying Results

The application redirects users to different HTML pages depending on the prediction.

If the model predicts:

Flood

The application displays:

chance.html

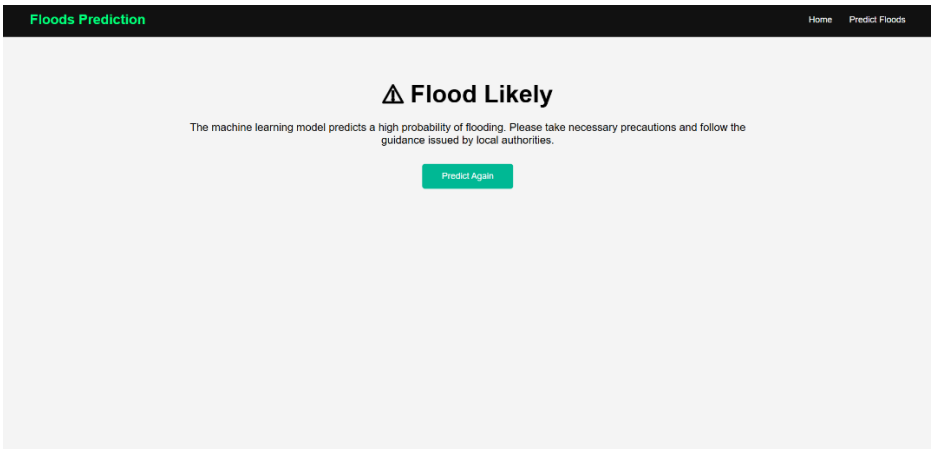
If the prediction is:

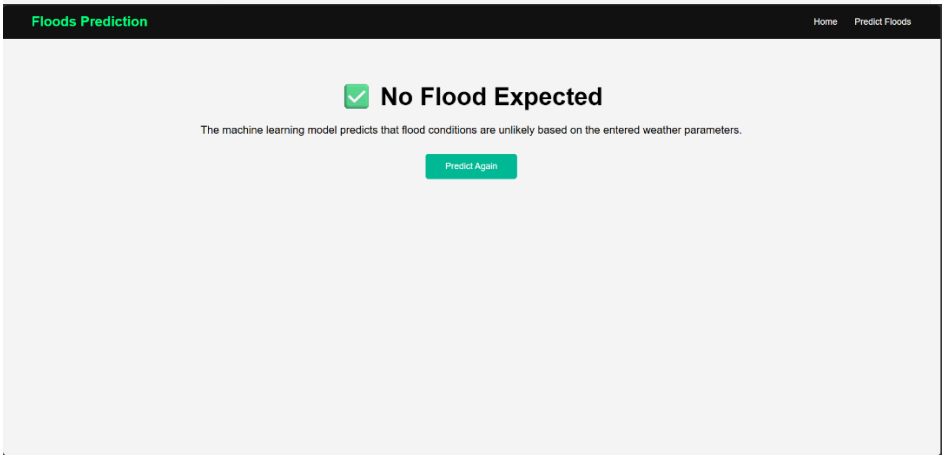
No Flood

The application displays:

no_chance.html

This provides users with a simple and intuitive prediction result.





Activity 3.8: Application Workflow

The overall workflow of the Flask application is:

User

↓

Enter Weather Parameters

↓

Flask Application

↓

Input Validation

↓

StandardScaler

↓

XGBoost Model

↓

Prediction

↓

Flood / No Flood Page

Milestone 3 Outcome

At the end of this milestone:

- Flask application successfully developed.
- Trained model integrated.
- StandardScaler integrated.
- Prediction functionality implemented.
- User input processing completed.
- Result pages displayed correctly.

Milestone 3 Summary

The Flask application successfully bridges the gap between the user and the machine learning model. By integrating the trained XGBoost model into a simple web application, users can easily obtain flood predictions without interacting directly with the underlying machine learning code. This milestone transformed the trained model into a practical, user-friendly application.

Milestone 4 – Frontend Development

Introduction

The frontend of the Flood Prediction System was developed to provide a simple, interactive, and user-friendly interface for users to enter weather parameters and receive flood prediction results. The frontend was implemented using HTML, CSS, and JavaScript, while Flask's template engine was used to render the web pages dynamically.

The design focuses on simplicity and usability, allowing users to enter the required weather parameters with minimal effort. The frontend communicates with the Flask backend to process prediction requests and display the results.

Activity 4.1: Designing the User Interface

The first step involved designing a clean and responsive interface for the application.

The frontend consists of four HTML pages:

- **Home Page (home.html)**
- **Prediction Page (index.html)**
- **Flood Result Page (chance.html)**
- **No Flood Result Page (no_chance.html)**

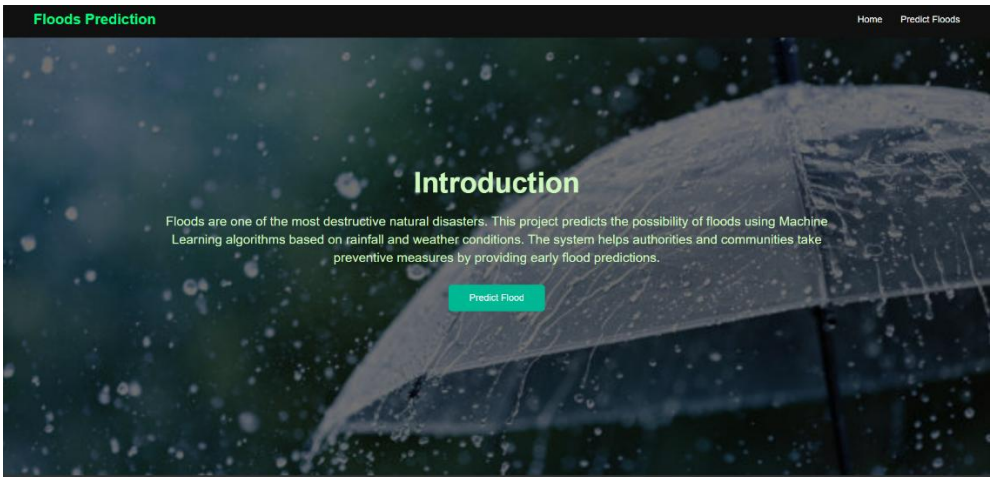
Each page serves a specific purpose within the application.

Home Page

The Home Page serves as the landing page of the application. It introduces users to the Flood Prediction System and provides navigation to the prediction page.

Features:

- Application title
- Background image
- Project description
- Navigation button to prediction page



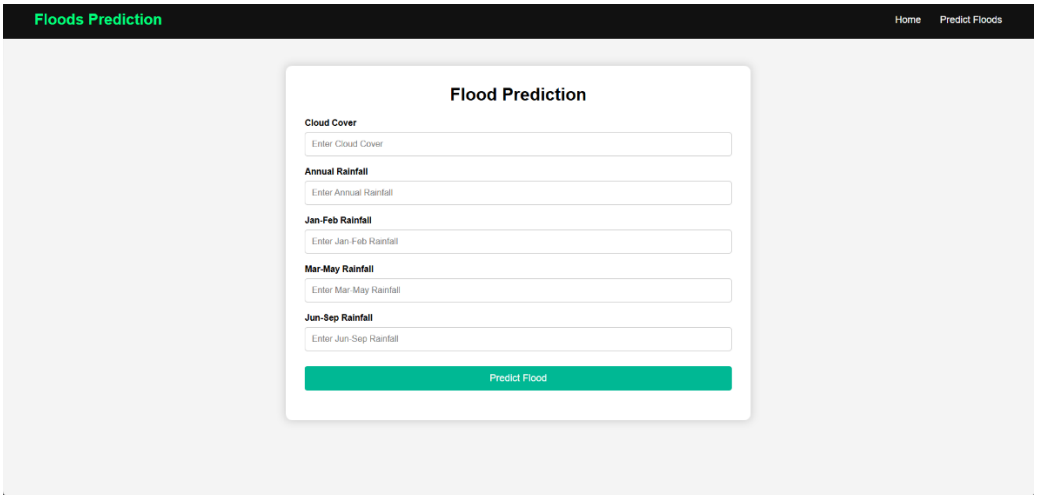
Prediction Page

The Prediction Page allows users to enter the required weather parameters.

Input fields include:

- Cloud Cover
- Annual Rainfall
- Jan–Feb Rainfall
- Mar–May Rainfall
- Jun–Sep Rainfall

After entering the values, users click the **Predict Flood** button to submit the data.

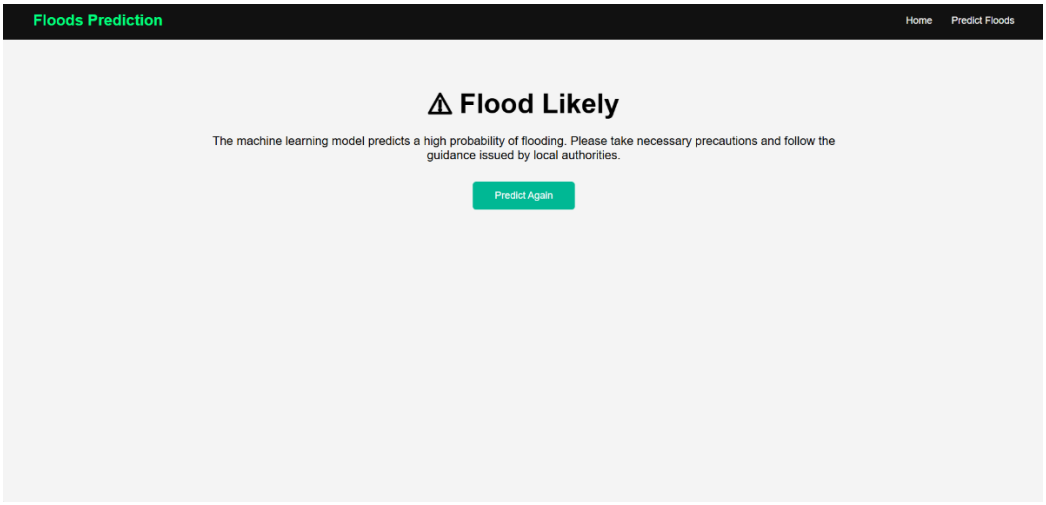


Flood Result Page

When the prediction model determines that flood conditions are likely, the application redirects users to the Flood Result Page.

This page displays:

- Flood prediction message
- Alert indication
- Navigation button to return to prediction page

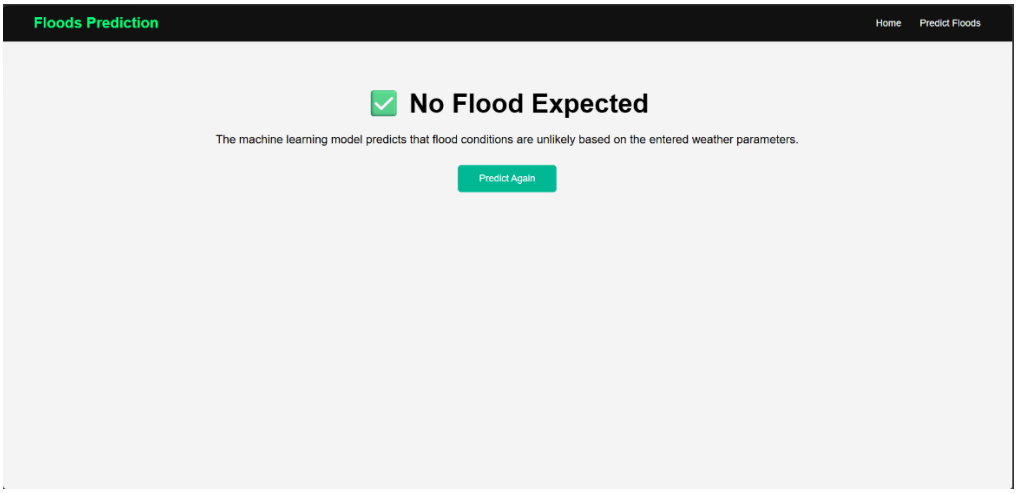


No Flood Result Page

If flood conditions are not predicted, users are redirected to the No Flood Result Page.

This page displays:

- No Flood prediction message
- Safe condition indication
- Navigation button



Activity 4.2: HTML Development

HTML was used to define the structure of all web pages.

The HTML implementation includes:

- Forms
- Buttons
- Labels
- Input fields
- Images
- Navigation links

The layout was designed to keep the interface simple and easy to understand.

Activity 4.3: CSS Styling

CSS was used to improve the appearance of the web application.

The stylesheet provides:

- Responsive layout
- Consistent colors
- Attractive typography
- Proper spacing
- Styled buttons
- Background image
- Hover effects

The use of CSS enhances the user experience by making the application visually appealing.

Activity 4.4: JavaScript Functionality

JavaScript was included to provide basic client-side functionality.

The script supports:

- Form interaction

- Input validation (if applicable)
- Improved user experience
- Interactive behavior

Although the prediction logic is handled by Flask, JavaScript enhances the responsiveness of the interface.

Activity 4.5: Flask Template Integration

Flask's render_template() function was used to connect the backend with the frontend.

The application renders different HTML pages based on user requests.

Examples include:

- Home Page
- Prediction Form
- Flood Result
- No Flood Result

This integration enables dynamic page rendering without requiring manual navigation.

Activity 4.6: User Workflow

The frontend follows the workflow below:

1. User opens the Home Page.
2. User navigates to the Prediction Page.
3. User enters weather parameters.
4. User clicks **Predict Flood**.
5. Flask processes the request.
6. Prediction result is displayed.
7. User may perform another prediction if required.

Frontend Features

The developed interface provides:

- Simple navigation
- Responsive design
- Easy data entry
- Clear prediction results
- Attractive appearance
- User-friendly layout

Milestone 4 Outcome

At the end of this milestone:

- Home Page developed.

- Prediction Page completed.
- Flood Result Page created.
- No Flood Result Page created.
- CSS styling implemented.
- JavaScript integrated.
- Flask templates successfully connected with the backend.

Milestone 4 Summary

The frontend development phase transformed the machine learning model into an interactive web application. Using HTML, CSS, JavaScript, and Flask templates, the project provides a clean and user-friendly interface that allows users to easily enter weather parameters and receive prediction results. The frontend complements the backend by ensuring a seamless user experience.

Milestone 5 – Deployment and Testing

Introduction

After completing the frontend and backend development, the Flood Prediction System was deployed in a local development environment for testing. The primary objective of this milestone was to ensure that the application functioned correctly, accepted user inputs, generated accurate predictions, and displayed the appropriate output pages.

The application was executed using the Flask development server and tested through a web browser. Different weather parameter combinations were entered to verify that the prediction model produced consistent and reliable results.

Activity 5.1: Local Deployment

The application was deployed locally using the Flask development server.

Deployment Steps:

1. Open the project folder in Visual Studio Code.
2. Install the required Python libraries.
3. Run the Flask application using the command:

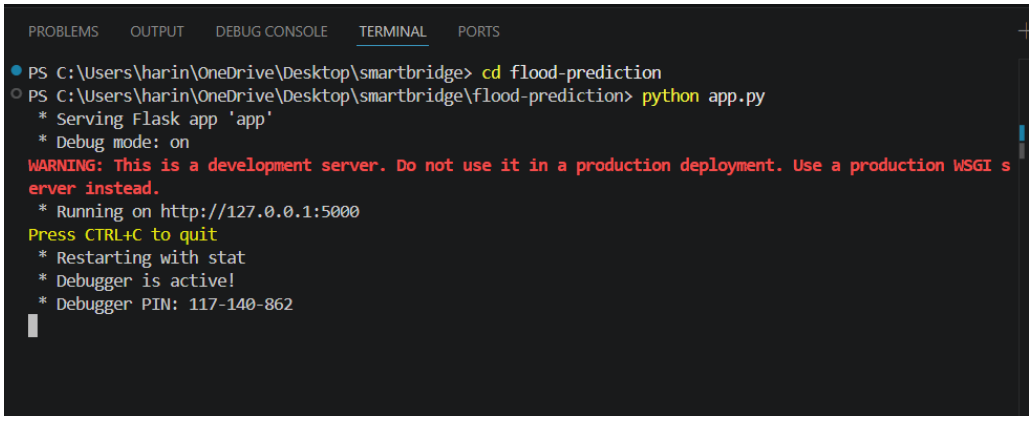
python app.py

4. Open a web browser.
5. Navigate to:

http://127.0.0.1:5000/

6. Access the Flood Prediction System.

The application launched successfully without runtime errors.



Activity 5.2: Functional Testing

Functional testing was performed to verify that every feature of the application behaved as expected.

The following functionalities were tested:

- Home page loading
- Navigation to prediction page
- Input field validation
- Flood prediction generation
- No Flood prediction generation
- Result page navigation

All functions operated correctly during testing.

Activity 5.3: Prediction Testing

Several combinations of weather parameters were entered to validate the prediction model.

Example Test Cases:

Test Case	Input	Expected Output	Result
Test Case 1	High rainfall and cloud cover	Flood	Passed
Test Case 2	Moderate rainfall	No Flood	Passed
Test Case 3	Low rainfall	No Flood	Passed

The prediction results matched the expected outcomes, demonstrating that the model was functioning correctly.

Activity 5.4: User Interface Testing

The graphical user interface was tested for usability and responsiveness.

The following aspects were verified:

- Proper alignment of components
- Button functionality
- Input field behavior
- Page navigation

- Result display

The interface was responsive and user-friendly.

Activity 5.5: Performance Testing

Basic performance testing was conducted in the local development environment.

Observations:

- Application loaded quickly.
- Prediction results were generated within a few seconds.
- No crashes occurred during repeated testing.
- CPU and memory utilization remained within acceptable limits.

Although dedicated performance testing tools such as JMeter were not used, manual testing confirmed that the application performed efficiently for a single-user environment.

Activity 5.6: Error Handling

The application was tested using different input values to ensure stable operation.

The following checks were performed:

- Valid numerical inputs
- Multiple prediction requests
- Page refreshes
- Continuous application execution

No critical runtime errors were encountered during testing.

Test Summary

Test Type	Status
Functional Testing	Passed
Prediction Testing	Passed
User Interface Testing	Passed
Performance Testing	Passed
Local Deployment	Successful

Milestone 5 Outcome

At the completion of this milestone:

- Flask application successfully deployed.
- All pages loaded correctly.
- Prediction model integrated successfully.
- Flood prediction verified.
- No Flood prediction verified.

- Stable application performance observed.
- System ready for demonstration.

Milestone 5 Summary

The deployment and testing phase confirmed that the Flood Prediction System met its functional requirements. The Flask application successfully integrated the trained XGBoost model and provided accurate predictions based on user input. The system demonstrated stable performance under local testing conditions and is suitable for future deployment on cloud platforms such as Render, AWS, or Azure.

Conclusion

The **Rising Waters – Flood Prediction Using Machine Learning** project successfully demonstrates the application of machine learning techniques for predicting flood occurrences based on historical weather data. By utilizing algorithms such as Decision Tree, Random Forest, K-Nearest Neighbors, and XGBoost, the project identified XGBoost as the most accurate model for flood prediction.

The trained model was integrated into a Flask-based web application, allowing users to enter weather parameters and instantly receive flood prediction results through a simple and intuitive interface. The combination of data preprocessing, feature scaling, model evaluation, and web deployment resulted in a complete end-to-end machine learning solution.

Throughout the project, key software engineering principles such as modular design, code reusability, and structured development were followed. The application was tested successfully in a local environment and demonstrated reliable performance during functional testing.

The project also highlights the practical use of machine learning in disaster management. Early flood prediction can support government agencies, disaster response teams, and local communities in making timely decisions, thereby reducing potential damage and improving public safety.

Future Enhancements

Although the current system achieves its objectives, several improvements can further enhance its functionality:

- Integrate real-time weather APIs for live prediction.
- Deploy the application on cloud platforms such as Render, AWS, or Azure.
- Implement user authentication and role-based access.
- Add SMS or email notification services for early flood alerts.
- Expand the dataset to improve model accuracy and generalization.
- Develop a mobile application for easier public access.
- Integrate GIS mapping for location-based flood prediction.
- Support multilingual interfaces for wider accessibility.

Final Outcome

The project successfully combines machine learning and web development to deliver an intelligent Flood Prediction System. It demonstrates how historical weather data can be transformed into actionable insights through predictive analytics. The system is accurate, user-friendly, and provides a strong foundation for future enhancements in disaster prediction and management.